perplexity

Покажи код для модуля model.py и metrics.py

Ниже — **полный код** для `model.py` и `metrics.py`, с учётом всех наших дополнений: нелинейности, обратные связи, задержки, пороги, адаптация весов, защита от усреднения и эрозии.

1. `src/les/core/model.py`

```
import numpy as np
from dataclasses import dataclass, field
from typing import List

@dataclass
class ModelParams:
    lambda_: np.ndarray = field(default_factory=lambda: np.array([0.5]*6))
    alpha: np.ndarray = field(default_factory=lambda: np.array([0.2]*6))
    beta: np.ndarray = field(default_factory=lambda: np.array([0.1]*6))
    gamma: np.ndarray = field(default_factory=lambda: np.array([0.1]*6))
    delta: np.ndarray = field(default_factory=lambda: np.array([0.05]*6))
    theta: np.ndarray = field(default_factory=lambda: np.array([0.3]*6))
    tau_E: float = 2.0
    delay: float = 0.5

class LivingEquilibriumModel:
    """
    Модель живого равновесия:
    S — выживание
    O — другие
    M — смысл
    P — процедура
    A — адаптация
    D — разнообразие (анти-усреднение)
    """

    def __init__(self, params: ModelParams):
        self.p = params

    def errors(self, x: np.ndarray) -> np.ndarray:
        return np.maximum(0, self.p.theta - x)

    def derivative(self, t: float, x: np.ndarray, x_delayed: np.ndarray = None) -> np
        S, O, M, P, A, D = x
        lam, alpha, beta, gamma, delta, theta = (
            self.p.lambda_, self.p.alpha, self.p.beta,
            self.p.gamma, self.p.delta, self.p.theta
        )
```

```
        e = self.errors(x)

        if x_delayed is None:
            x_delayed = x

        S_d, O_d, M_d, P_d, A_d, D_d = x_delayed
        x_bar = np.mean(x[:5])

        dS = -lam[^0]*S + alpha[^0]*e[^0] + beta[^0]*O_d*M_d - gamma[^0]*P_d - delta
        dO = -lam[^1]*O + alpha[^1]*e[^1] + beta[^1]*S_d*M_d - gamma[^1]*P_d - delta
        dM = -lam[^2]*M + alpha[^2]*e[^2] + beta[^2]*S_d*O_d - gamma[^2]*P_d - delta
        dP = -lam[^3]*P + alpha[^3]*e[^3] + beta[^3]*S_d*O_d*M_d - gamma[^3]*A_d - de
        dA = -lam[^4]*A + alpha[^4]*e[^4] + beta[^4]*P_d - gamma[^4]*(S+O+M) + delta
        dD = -lam[^5]*D + alpha[^5]*e[^5] + beta[^5]*(S+O+M) - gamma[^5]*x_bar

        return np.array([dS, dO, dM, dP, dA, dD])

    def adapt_weights(self, w: np.ndarray, e: np.ndarray, alpha: float = 0.1) -> np.n
        return w + alpha * e

    def cumulative_erosion(self, errors_history: List[np.ndarray]) -> float:
        return np.sum(errors_history)

    def should_stop(self, E: float) -> bool:
        return E > self.p.tau_E
```

2. `src/les/core/metrics.py`

```
import numpy as np
import pandas as pd
from typing import Dict

class MetricsCalculator:
    def __init__(self, df: pd.DataFrame, thresholds: np.ndarray):
        self.df = df
        self.thresholds = thresholds

    def summary(self) -> Dict[str, dict]:
        metrics = {}
        for col in ['S', 'O', 'M', 'P', 'A', 'D']:
            series = self.df[col]
            metrics[col] = {
                'mean': series.mean(),
                'std': series.std(),
                'min': series.min(),
                'max': series.max(),
                'q25': series.quantile(0.25),
                'q50': series.quantile(0.50),
                'q75': series.quantile(0.75),
                'danger_points': int((series < self.thresholds[^0]).sum()),
            }
        return metrics

    def diversity_index(self) -> float:
        return self.df['D'].mean()
```

```
    def erosion_index(self) -> float:
        errors = np.maximum(0, self.thresholds[^0] - self.df[['S','O','M','P','A','D
        return errors.sum()

    def report(self) -> str:
        metrics = self.summary()
        lines = ["ОТЧЁТ ПО МОДЕЛИ ЖИВОГО РАВНОВЕСИЯ", "="*40, ""]
        for k, v in metrics.items():
            lines.append(f"Контур {k}:")
            for mk, mv in v.items():
                lines.append(f"  {mk}: {mv:.4f}")
            lines.append("")
        return "\n".join(lines)
```

3. Как использовать

```
from les.core.model import LivingEquilibriumModel, ModelParams
from les.core.metrics import MetricsCalculator
import pandas as pd

params = ModelParams()
model = LivingEquilibriumModel(params)

# Пример данных
df = pd.DataFrame({
    'S': [0.8, 0.7, 0.6],
    'O': [0.8, 0.7, 0.6],
    'M': [0.8, 0.7, 0.6],
    'P': [0.8, 0.7, 0.6],
    'A': [0.8, 0.7, 0.6],
    'D': [0.8, 0.7, 0.6],
})

calc = MetricsCalculator(df, params.theta)
print(calc.summary())
print(calc.report())
```

Что реализовано

- `model.py`: нелинейности, обратные связи, задержка, пороги, адаптация весов, эрозия.
- `metrics.py`: описательная статистика, индекс разнообразия, индекс эрозии, текстовый отчёт.

Финальная формулировка

> **`model.py` — это «сердце» системы, а `metrics.py` — её «зрение»: одно моделирует, другое измеряет.**

Это и есть основа для дальнейшего расширения: агенты, рой, визуализация, оптимизация.[1] [2] [3] [4]

⁂

1. https://pandas.pydata.org/pandas-docs/dev/reference/api/pandas.DataFrame.describe.html
2. https://pandas.pydata.org/pandas-docs/version/3.1/getting_started/intro_tutorials/06_calculate_statistics.html
3. https://docs.scipy.org/doc//scipy-1.18.0/reference/generated/scipy.integrate.solve_ivp.html
4. https://github.com/scipy/scipy/blob/v1.16.1/scipy/integrate/_ivp/ivp.py
5. https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.info.html
6. https://pandas.pydata.org/pandas-docs/version/0.19/generated/pandas.DataFrame.describe.html
7. https://library.virginia.edu/data/articles/getting-started-with-pandas-in-python
8. https://train.rse.ox.ac.uk/material/HPCu/introductory_courses/python/07_pandas_dataframes
9. https://pandas.pydata.org/pandas-docs/stable/getting_started/intro_tutorials/06_calculate_statistics.html
10. https://earthdatascience.org/courses/intro-to-earth-data-science/scientific-data-structures-python/pandas-dataframes/run-calculations-summary-statistics-pandas-dataframes/
11. https://pandas.pydata.org/pandas-docs/version/2.2/reference/api/pandas.DataFrame.describe.html
12. https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.describe.html
13. https://pandas.pydata.org/pandas-docs/stable/dev/reference/api/pandas.DataFrame.describe.html
14. https://docs.scipy.org/doc//scipy-1.3.1/reference/generated/scipy.integrate.solve_ivp.html
15. https://pandas.pydata.org/docs/getting_started/intro_tutorials/06_calculate_statistics.html
16. https://www.annenbergclassroom.org/glossary_term/majority-rule-and-minority-rights/
17. https://www.britannica.com/topic/veche
18. https://www.victorianweb.org/victorian/philosophy/mill/liberty.html
19. https://en.wikipedia.org/wiki/On_Liberty